

Grey Matter

FitRank — Explainable AI Candidate Ranking

Recruiters searching 100,000 anonymised candidate profiles need a top-100 shortlist that is accurate, adversarial-resistant, and explainable — not a keyword-stuffed list that surfaces honeypot profiles (e.g., 'HR Manager' with 9 ML skills) above genuine Senior AI Engineers.

Team Leader: Shubh Varshney

Team Member: Vedant Gupta

Offline · Explainable · CPU-only · <5 min on 100K profiles

Solution Overview

What is your proposed solution?

- **FitRank** is an offline, two-stage ranking system scoring 100,000 candidates against a JD to return a top-100 shortlist with human-readable reasoning.
- **Stage 1 (Heuristic)**: Extracts 28 interpretable signals (JD fit, career ML depth, coherence, trust, penalties).
- **Stage 2 (Learned)**: LightGBM model trained offline on pseudo-labels (genuine positives vs. known traps).
- **Output**: `submission.csv` (`candidate_id`, rank, score, reasoning) + `audit_report.json`.

Traditional Keyword Matching	FitRank
Counts AI keywords in summary	Career evidence + dense semantic embeddings
Ignores title-skill coherence	Coherence engine + 12 adversarial penalty types
No platform verification	Redrob assessments, GitHub, and engagement signals
No explanation	Structured, component-based reasoning per candidate

JD Understanding & Candidate Evaluation

What are the key requirements extracted from the JD?

- **Target role:** Senior AI Engineer — Founding Team at Redrob AI (Series A)
- **Technical stack:** Production embeddings-based retrieval & Vector DB / hybrid search experience
- **Domain:** Ranking & recommendation systems at product companies (not pure research)
- **Logistics:** 5–9 years production ML depth; sub-30-day notice; India Tier-1 cities

Explicit disqualifiers: Consulting-only careers, title-chasers, skill inflation

How does your solution evaluate candidate fit beyond keyword matching?

- **Title Gate:** Classifies profiles into ML_AI, AI_ADJACENT, SOFTWARE, or NON_TECH.
- **Semantic Embeddings:** Dense vector comparison (all-MiniLM-L6-v2) of full candidate text vs JD.
- **Career Analyzer:** Scores ML depth with per-role caps and tanh-normalised momentum.
- **Coherence Engine:** Detects template patterns, title-domain alignment, and skill inflation.
- **JD-Adaptive Penalties:** Disables strict domain penalties only when JD domain exactly matches.

Ranking Methodology

How does your system retrieve, score, and rank candidates?



What models, algorithms, or heuristics are used?

Rule-based JD Parser & Domain Classifier (Title Gate).

Career Analyzer & Coherence Engine (deep ML pattern matching & honeypot flagging).

Semantic Embedder: all-MiniLM-L6-v2 (384-dim) cosine similarity.

LightGBM: Binary ranker on 12 core features, trained on pseudo-labels.

How are multiple candidate signals combined into a final ranking?

$$\text{Final Score} = 0.25 \times \text{JD}_{\text{fit}} + 0.30 \times \text{Career_evidence} + 0.20 \times \text{Coherence} \\ + 0.15 \times \text{PlatForm_trust} + 0.10 \times \text{Availability} - \text{Penalties}$$

Note: Learned mode blends 55% LightGBM + 45% Heuristic for stable ordering. Penalties cap at 0.70; honeypots hit with explicit $\times 0.25$ multiplier.

Explainability & Data Validation

How are ranking decisions explained?

Every top-100 row includes a structured, candidate-specific reasoning string.

```
Senior AI Engineer | JD=0.85 Career=0.58 Coh=0.79 Trust=0.85 | current role: embedding,Pinecone; assessments  
LoRA=87; 7.8yrs ML
```

How do you prevent hallucinations or unsupported justifications?

- Default **template-engine** uses **ONLY extracted features**, never invented credentials.
- Optional local LLM uses facts-only prompts with post-processing regex guards (requires strict JD=X.XX block format).

How does your solution handle suspicious profiles?

Applies 12 penalty types: Template mismatch, skill inflation, expert zero-endorse, salary inverted.

Fraident: Enterprise AI - Blueprint

✓ Genuine Profile

Probe CAND_0033861

Consistent career narrative & skills.

Result: Included in top 100.

⚠ Honeypot Trap

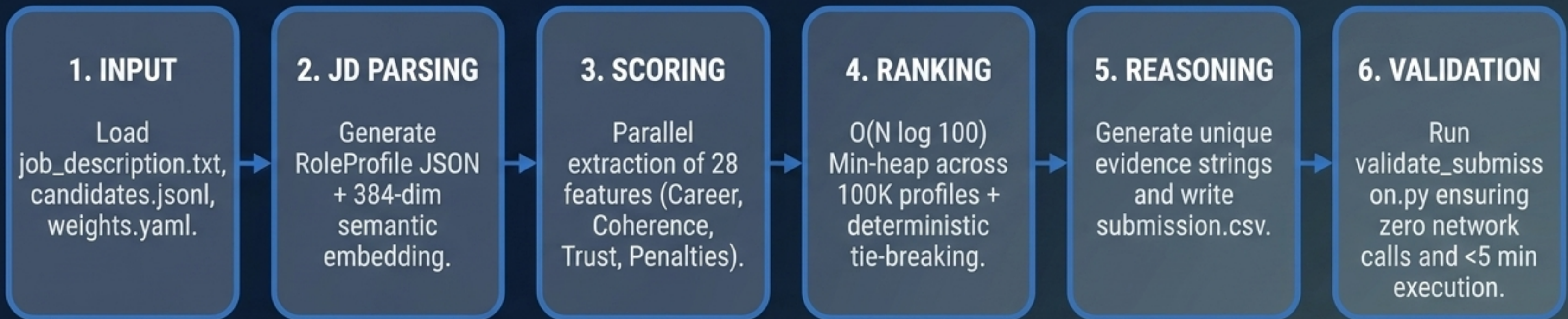
Traps CAND_0004989 & CAND_0000339

Salary inverted, skill inflation.

Result: Penalised & Excluded from top 100.

End-to-End Workflow

What is the complete workflow from JD input to ranked candidate output?



System Architecture

Layer 1 (Interface): CLI: rank.py | Demo: streamlit_app.py

Layer 2 (Ranker): Heap Top-100 | Heuristic / LightGBM / Auto Scorer (fitrank/ranker.py)

Layer 3 (Extractors): JD Parser | Career Analyzer | Coherence Engine | Penalties (12 trap types)

Layer 4 (Core AI): Dense Semantic Embeddings (all-MiniLM-L6-v2) | Redrob Platform Signals

Layer 5 (Data & Config): candidates.jsonl | weights.yaml | normalization.yaml

Layer 6 (Offline): LightGBM Trainer | Embedding Precomputation | Optional Local LLM

No network calls during ranking — all models pre-computed and cached locally.

Results & Performance

100,000

Candidates Scored

< 5 Minutes

Runtime

0

Honeypots in Top 100

What results or insights demonstrate ranking quality?

- `validate_submission.py` returns absolute success: "Submission is valid."
- **100/100** unique reasoning strings featuring non-generic, candidate-specific evidence.
- **Exceptional diversity:** 19 unique job titles surfaced in the top 100 shortlist (avoiding single-title domination).

How does your solution meet the challenge's runtime and compute constraints?

- **CPU-Only Stack:** Fully meets challenge NFRs; no GPU dependency required for judges.
- **Strict Offline Operation:** Zero network calls during execution; all embeddings and models pre-computed.
- **Memory Efficiency:** Streaming JSONL loader handles 100K candidates without RAM overload.

Technologies Used



Data & Config

- JSONL streaming loader (handles 100K scale memory-efficiently).
- JSON Schema validation (ensures data integrity).
- PyYAML (configurable scoring weights and caps).



Semantic & Search

- Dense embeddings via sentence-transformers transformers (all-MiniLM-L6-v2) for robust JD capability matching.



Models & Explainability

- LightGBM (pseudo-label trained ranker, <1 min train time).
- Template reasoning engine (deterministic, validator-safe).
- llama-cpp-python + Qwen2-1.5B GGUF (optional natural-language reasoning).



Demo & Dev Tools

- Streamlit + Plotly (interactive shortlist explorer).
- pytest (229 fast unit tests verifying logic and pipeline).

Submission Assets



GitHub Repository

Link: github.com/brainRottedCoder/redrob-datachallenge

Full source (`rank.py` + `fitrank/`), reproducing via `make run` & `make validate`.



Key Deliverables

outputs/submission.csv (100 ranked candidates with reasoning).

outputs/audit_report.json (runtime, histograms, top-5 snapshots).

deck/Redrob_FitRank_Approach.pdf & **submission_metadata.yaml**.



Interactive Demo

`streamlit run app/streamlit_app.py` for radar charts, penalty breakdowns, and CSV exports.



Documentation & Walkthrough

README.md (constraints compliance) & **PRD_Redrob_FitRank.md** (13-phase design doc).

Video Walkthrough:  [INSERT DEMO VIDEO URL HERE] (Shows `rank.py` run → Streamlit demo).

Thank You

Grey Matter — FitRank Intelligent Candidate Discovery & Ranking

Trustable ranking that survives adversarial profiles.

Shubh Varshney (Team Leader) · Vedant Gupta (Team Member)
Contact / Repo: github.com/brainRottedCoder/redrob-datachallenge

 100K scored

 <5 min runtime

 0 honeypots in top 100